

Supplementary Material S2

Evaluation module: spatial K -fold construction, secondary-mesh adjacency, and leave-prefecture-out bounding boxes

R. Okauchi P.-J. Chun

May 20, 2026

1 Purpose

This document specifies the evaluation-side routines whose *names* are referenced by the body of the paper but whose implementation details would interrupt the methods narrative. The canonical implementations of each routine are released under the paper companion repository [kanto-calibrated-spt-prior](#) (directory `code/evaluation/`), under the MIT license.

Three routines are documented here:

1. **secondary_mesh_adjacency** — 8-neighbour adjacency of Japanese government *secondary* mesh cells; used by the buffered spatial cross-validation protocol (Section 4.4 of the paper).
2. Spatial K -fold assignment generators — the random load-balanced and the contiguous k-means protocols (Section 2.6 of the paper).
3. **prefecture_regions** — bounding boxes used by the Leave-prefecture-out protocol (Section 4.5 of the paper).

2 Secondary-mesh adjacency

Japanese government *secondary mesh* codes (also known as “2nd-level standard regional mesh”) tile the country into $5'$ latitude $\times$ $7'30''$ longitude cells. A code has six digits: the first four identify the *primary* mesh (1° latitude $\times$ $1^\circ 30'$ longitude) and the last two encode the row and column within that primary mesh.

For mesh-buffered spatial cross-validation we need, for a given secondary mesh, its 8 neighbours (the surrounding ring). The following pseudo-code implements this without round-trip geographic conversion:

```
def adjacent_secondary_mesh_codes(code: str) -> list[str]:
    """Return up to 8 secondary-mesh codes adjacent to 'code'."""
    assert len(code) == 6 and code.isdigit()
    p_lat = int(code[0:2])      # primary mesh latitude index
    p_lon = int(code[2:4])      # primary mesh longitude index
    s_row = int(code[4])        # secondary row inside primary (0-7)
    s_col = int(code[5])        # secondary column inside primary (0-7)

    neighbours = []
    for d_row in (-1, 0, 1):
        for d_col in (-1, 0, 1):
            if d_row == 0 and d_col == 0:
                continue
            new_row = s_row + d_row
```

```

new_col = s_col + d_col
new_p_lat = p_lat
new_p_lon = p_lon

# Carry across primary-mesh boundaries.
if new_row < 0:
    new_row += 8
    new_p_lat -= 1
elif new_row > 7:
    new_row -= 8
    new_p_lat += 1
if new_col < 0:
    new_col += 8
    new_p_lon -= 1
elif new_col > 7:
    new_col -= 8
    new_p_lon += 1

if 0 <= new_p_lat < 100 and 0 <= new_p_lon < 100:
    neighbours.append(
        f"{new_p_lat:02d}{new_p_lon:02d}"
        f"{new_row}{new_col}"
    )
return neighbours

```

The implementation in the companion repository adds defensive checks for edge cases at the boundaries of the Japanese mesh system (*e.g.* Hokkaido — Okinawa wraps that are geographically meaningless) and short-circuits when a candidate neighbour is not present in the active dataset. Both versions return the same answer for Kanto-internal codes.

3 Spatial K -fold assignment

3.1 Random load-balanced protocol

The random load-balanced protocol assigns every secondary-mesh code to one of K folds with the goal of *approximate row balance* rather than spatial contiguity. The procedure is:

1. Enumerate all distinct secondary-mesh codes that contain at least one boring in the Kanto extract.
2. Sort meshes by descending row count to make the bin-packing step well-conditioned.
3. Walk the sorted list and place each mesh into the fold with the smallest current row count (greedy first-fit- decreasing). Ties are broken by the lowest fold index.
4. Persist the resulting (mesh_code, fold_idx) table as fold_indices/kanto_random_K_seed42.csv in the companion repository.

For $K = 3$ and seed 42 this yields fold sizes 165,096/166,884/163,745 rows (see Section 5.1 of the paper).

3.2 Contiguous k-means protocol

The contiguous protocol clusters mesh *centroids* into K geographic blocks using k-means in (lat, lon) space:

1. Compute the centroid $(\bar{\phi}_m, \bar{\lambda}_m)$ of every mesh m in the Kanto extract.

2. Run k-means with K clusters and seed 42 on the centroid coordinates.
3. Assign each mesh to its cluster as its fold index.
4. Persist as `fold_indices/kanto_contig_K_kmeans.csv`.

This yields three geographically contiguous folds (*cf.* Figure 1(c) of the paper).

4 Leave-prefecture-out bounding boxes

The Leave-prefecture-out protocol (Section 4.5 of the paper) holds out an entire Kanto prefecture from training and evaluates on all of its boreholes. Three prefectures are reported in the paper (Tokyo, Tochigi, Chiba); a bounding box is used in lieu of the full administrative polygon because the operationally meaningful quantity is SPT density inside a region, not its political boundary.

The bounding boxes are (latitude, longitude in degrees, EPSG:4326):

Prefecture	Lat range	Lon range
Tokyo	35.49 – 35.90	139.30 – 139.93
Tochigi	36.20 – 37.16	139.31 – 140.30
Chiba	34.90 – 36.13	139.74 – 140.87

The companion repository’s `code/evaluation/ prefecture_regions.py` provides the same table in machine-readable form.

5 Notes

- All routines documented here are pure functions of the secondary-mesh ID table and the prefecture bounding-box table; they do not depend on the trained model. They are therefore stable across retraining and can be reused by external benchmarks.
- The buffered CV protocol of Section 4.4 of the paper constructs a 1-mesh ring around every test mesh by invoking `adjacent_secondary_mesh_codes` once per test mesh; the resulting set is excluded from training. Under the contiguous fold geometry this removes $\sim 16\%$ of training rows; under the random fold geometry it removes $\sim 93\%$, because adjacent meshes are frequently assigned to different folds. The body of the paper interprets the random-buffer cell as a worst-case spatial-leak upper bound, not a clean leakage diagnostic.